Santiago Giraldo
Katherine Quiceno
Stephanie Vargas

1. BACKEND IMPLEMENTED

1.1. Direction Cosine Matrix (DCM)

The transformation from the body frame (X forward, Y right, Z down) to the NED frame is performed using the Direction Cosine Matrix:

$$V^{NED} = C_b{}^n \cdot V_{body}$$

The matrix is constructed from the Euler angles (φ, θ, ψ) using the 3-2-1 rotation sequence (yaw-pitch-roll). The implementation is contained in the *build_dcm()* function within the flight_math.py module, where input angles are converted from degrees to radians for trigonometric computation.

$$C_b{}^n = \begin{bmatrix} cos\theta cos\varphi & cos\theta sen\varphi & -sen\theta \\ sen\phi sen\theta cos\psi - cos\phi sen\varphi & sen\phi sen\theta sen\psi + cos\theta cos\varphi & sen\phi cos\theta \\ cos\phi sen\theta cos\varphi + sen\phi sen\varphi & cos\phi sen\theta sen\varphi - sen\phi cos\varphi & cos\phi cos\theta \end{bmatrix}$$

1.2. Velocity Transformation

The *transform_flight_data()* function computes the NED velocity components$[V_N, V_E, V_D]$ from the body velocity vector $[u, v, \omega]$ using the DCM. Proper vector reshaping is handled internally to ensure correct matrix multiplication.

1.3. 3D Visualization

A simplified aircraft wireframe is defined in body coordinates and transformed into the NED frame using the same DCM. The visualization includes:

- NED reference axes (red=North, green=East, blue=Down).
- Scaled velocity vector (magenta).
- Aircraft geometry (nose, tail, wingtips, vertical stabilizer).

1.4. Flight Instruments

Two cockpit-style instruments were implemented:

- Attitude Indicator: Displays pitch (vertical displacement of horizon) and roll (rotation of horizon), including bank angle markings at 0°, 10°, 20°, 30°, 45°, and 60°.
- Heading Indicator: Compass with marks every 30° and N/E/S/W letters. An orange arrow indicates the current heading (ψ).

Both instruments use circular clipping to simulate realistic cockpit displays.

2. AERODYNAMIC ANGLES CALCULATION

The aerodynamic angles are computed in the *compute_aero_angles()* function using the body-axes velocity components (u, v, ω).

2.1. Angle of Attack (α)

$$\boldsymbol{\alpha = tan^{-1}\left(\frac{\omega}{u}\right)}$$

Defines the angle between the longitudinal body axis and the relative wind in the body X-Z plane.

2.2. Sideslip Angle (β)

$$\beta = sen^{-1}\left(\frac{v}{V}\right) \qquad \text{with} \qquad V = \sqrt{u^2 + v^2 + \omega^2}$$

Representing the lateral deviation of the relative wind from the longitudinal axis, positive when the airflow comes from the right.

2.3. Flight Path Angle (γ)

$$\gamma = \theta - \alpha$$

Vaid under the small-sideslip assumption (β=0). A conditional check is included to handle the stationary case (V=0), preventing division by zero and returning null aerodynamic angles.

3. OBSERVATIONS FROM FLIGHT CASES

Three representative flight cases were evaluated to validate the kinematic model and aerodynamic angle computation.

3.1. Case A-Straight and Level Flight

With zero roll and pitch and velocity aligned with the body X-axis, the NED velocity distributes equally between North and East due to $\psi = 45°$. All aerodynamic angles are zero, confirming consistency under trimmed, symmetric conditions.

3.2. Case B-Climb

With $\theta = 15°$ and velocity aligned with the body axis, the vertical NED component becomes negative (Down-positive convection), indicating climb. Since α=0°, the flight path angles satisfy $\gamma = \Theta$, validating the relation $\gamma = \theta - \alpha$.

3.3. Case C-Climbing Right Turn

Introducing roll, yaw, and non-zero lateral and vertical body velocities produce non-zero aerodynamic angles (α=11°, β=3°). The positive angle of attack indicates lift generation, while the small sideslip reflects lateral motion. Despite positive pitch, γ becomes negative due to the combined effect of α and velocity components, resulting in a net descending trajectory.

3.4. Cross-Validation

The velocity magnitude remains invariant between body and NED frames, confirming that the implemented DCM is orthonormal and the transformation is mathematically consistent.

The code can be found in this GitHub repository:

https://github.com/SantiagoGiraldoM17/Flight_Dynamics